Mastering Mobile Manipulation with LIMO-Robot

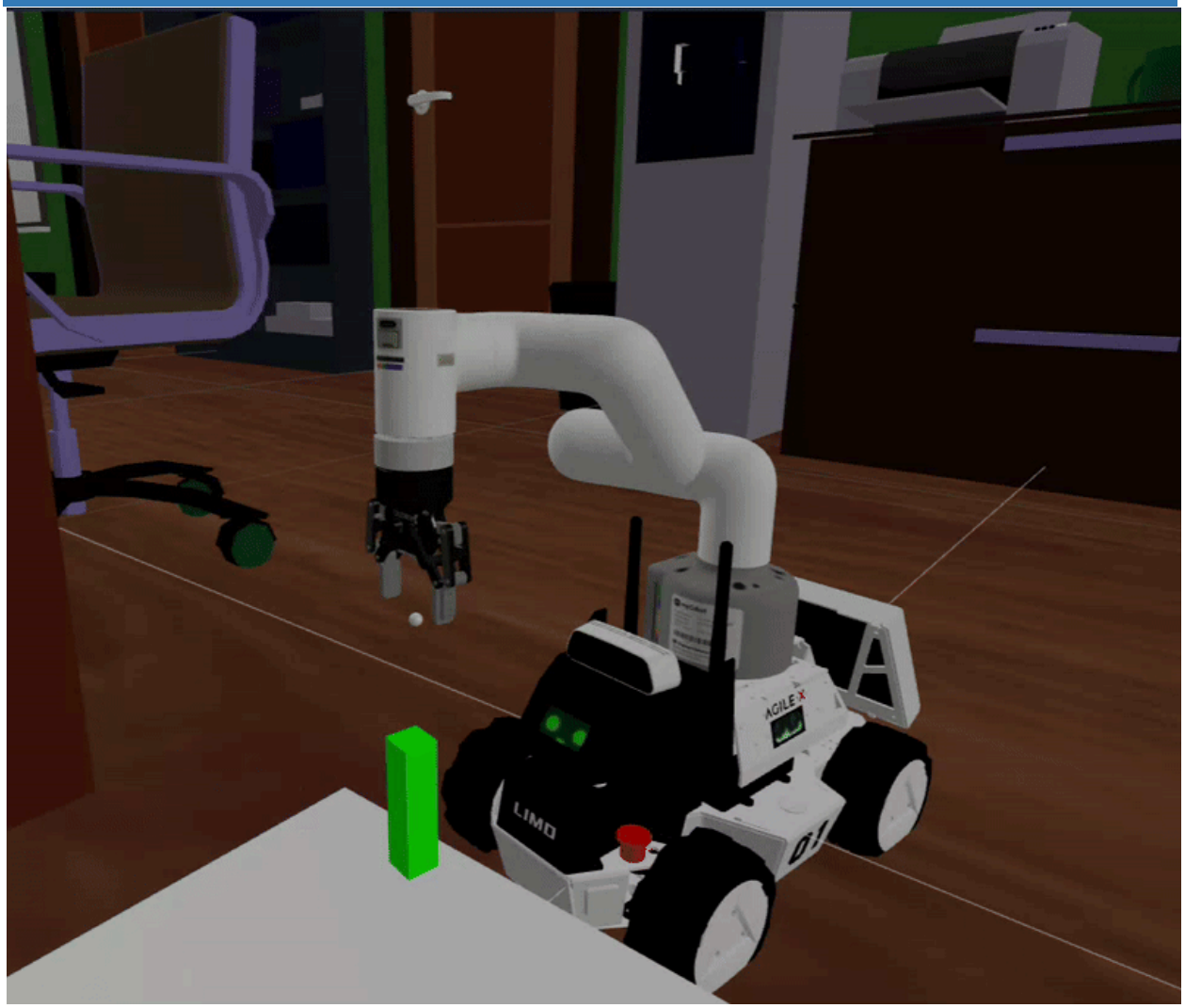
Unit 2: Doing Motion Planning Programmatically

- Summary -

Estimated time for completion: **4 hours**

This unit shows you how to perform motion planning using the **Move Group C++ Interface**. You will learn to create a C++ program communicating with MoveIt2 functionalities to perform motion planning.

- End of Summary -



In the previous unit, you saw that you could plan and execute trajectories for your robot using the MoveIt2 RVIZ environment. However, this is not the typical case.

Usually, you want to move your robot with your own programs - this is what you will do in this unit! However, you will use C++ to control the robot in this course. The reason is that MoveIt2 did not have a Python API available when writing this tutorial.

2.1 Planning a Trajectory

In motion planning, you can differentiate between two main basic methods:

1. **Planning to a joint-space goal**: In this method, set the desired joint positions for the arm. The motion planning algorithm computes a trajectory from the current joint positions to the new desired ones. This motion planning method is easier as it does not involve Inverse Kinematics (IK).
2. **Planning to an end-effector pose**: In this approach, set the desired pose, which includes both position and orientation, for the end effector link in space. This motion planning method is more complex as it involves IK.

2.1.1 Planning to a joint-space goal

As you have seen in the previous unit, **there is a difference between planning and executing a trajectory**. In the first part of the unit, learn how to plan a trajectory with C++. In particular, see how to plan to a joint-space goal.
Review the code you will be executing:
test_trajectory.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
```

```
                                        PLANNING_GROUP);

// We set parameters
move_group.setMaxVelocityScalingFactor(1.0);
move_group.setMaxAccelerationScalingFactor(1.0);
move_group.setPlanningTime(10.0);
move_group.setPoseReferenceFrame("base_footprint");

const moveit::core::JointModelGroup *joint_model_group =
   move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

RCLCPP_INFO(LOGGER, "Planning frame: %s",
           move_group.getPlanningFrame().c_str());

RCLCPP_INFO(LOGGER, "End effector link: %s",
           move_group.getEndEffectorLink().c_str());

RCLCPP_INFO(LOGGER, "Available Planning Groups:");
std::copy(move_group.getJointModelGroupNames().begin(),
         move_group.getJointModelGroupNames().end(),
         std::ostream_iterator<std::string>(std::cout, ", "));

moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

std::vector<double> joint_group_positions;
current_state->copyJointGroupPositions(joint_model_group,
                                       joint_group_positions);

// Go Home movement
std::vector<double> home_position = {-1.57, 0.0, 0.7, 0.0, 0.0, 0.0};
move_group.setJointValueTarget(home_position);

moveit::planning_interface::MoveGroupInterface::Plan my_plan;

bool success =
   (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

if (success == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
  RCLCPP_INFO(LOGGER, "Moved to home position successfully.");
} else {
  RCLCPP_ERROR(LOGGER, "Failed to move to home position.");
  return 1;
}

rclcpp::shutdown();
return 0;
```

```
}
```

That is great! However, how does this code work? What does each part mean? Break it down into smaller pieces.
First, define a couple of statical variables for the **logger** system and **your arm's Planning Group**, named `ur_manipulator`:

In [ ]:

```
static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP_ARM = "ur_manipulator";
```

Here, create a ROS2 node and add this node to a `SingleThreadedExecutor` executor. Then, spin this executor until somebody terminates the program.

In [ ]:

```
auto move_group_node =
    rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();
```

Next, create the `MoveGroupInterface` and set the parameters:

In [ ]:

```
moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                          PLANNING_GROUP);

  // We set parameters
  move_group.setMaxVelocityScalingFactor(1.0);
  move_group.setMaxAccelerationScalingFactor(1.0);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");

  const moveit::core::JointModelGroup *joint_model_group =
    move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);
```

You can see that you initialize a **`MoveGroupInterface`** named **`move_group`** for the arm Planning Group. It can be easily set up using the name of the Planning Group you would like to control and plan for. This interface allows you to call the Move Group node capabilities from a C++ program. Next, set parameters that will help you:

- `MaxVelocityScalingFactor`: This parameter sets the percentage of the maximum speed the joints can achieve. In this case, it is set to 1.0, meaning 100%. Based on the joint limits, this implies that the arm will move as fast as possible.

- `MaxAccelerationScalingFactor`: This parameter determines how much of the acceleration limit you adhere to. In this case, we allow the maximum acceleration.
- `PlanningTime`: By setting longer times in seconds here, systems with lower resources that are slower in planning trajectories can succeed without timing out.
- `PoseReferenceFrame`: You specify in which frame MoveIt will consider the coordinates you send it. This is especially important when setting the tool's position and orientation.

Next, create a **`JointModelGroup`** pointer:

In [ ]:

```
const moveit::core::JointModelGroup *joint_model_group =
    move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);
```

Pointers are often used to refer to the Planning Group for improved performance. This `JointModelGroup` pointer contains the information related to the Planning Group.
Next, get and print information related to your setup:

In [ ]:

```
// We print the current planning frame for this group.
RCLCPP_INFO(LOGGER, "Planning frame: %s",
            move_group.getPlanningFrame().c_str());

// We can also print the name of the end-effector link for this group.
RCLCPP_INFO(LOGGER, "End effector link: %s",
        move_group.getEndEffectorLink().c_str());

// We can get a list of all the groups in the robot:
RCLCPP_INFO(LOGGER, "Available Planning Groups:");
std::copy(move_group.getJointModelGroupNames().begin(),
      move_group.getJointModelGroupNames().end(),
      std::ostream_iterator<std::string>(std::cout, ", "));
```

Here, you get data about your MoveIt2 setup. This is not mandatory for planning, but it can be helpful to ensure everything is configured as it should. Specifically, you are getting data about the following:

- The name of the frame in which the robot is planning
- The current end effector link
- The available Planning Groups

Next, you get the **`current state`** of your robot:

In [ ]:

```
moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

std::vector<double> joint_group_positions;
current_state->copyJointGroupPositions(joint_model_group,
                                       joint_group_positions);
```

- Here, you want to get the current state of the robot.
- The **parameter `10` indicates the maximum wait time (seconds)** to get the current state. This is a timeout to ensure the program does not hang if the state is not available promptly. It tries to ensure the state information is as fresh as possible, critical for accurate movement planning.
- Then, copy the values of the variable **`joint_model_group`** (which contains the position value of each of the robot's joints) into the vector **`joint_group_positions`**.

Finally, plan a motion by changing the position values of the joints:

In [ ]:

```
// Go Home movement
std::vector<double> home_position = {-1.57, 0.0, 0.7, 0.0, 0.0, 0.0};
move_group.setJointValueTarget(home_position);

moveit::planning_interface::MoveGroupInterface::Plan my_plan;

bool success =
  (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

if (success == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
RCLCPP_INFO(LOGGER, "Moved to home position successfully.");
} else {
RCLCPP_ERROR(LOGGER, "Failed to move to home position.");
return 1;
}
```

Here, you are doing a couple of essential things:

- First, set the joint's desired position values. In this case, set the joint values of the modified **home** position, where you turn the first joint `-1.57` and the third joint `0.7` radians.
- Second, these new values are set as the new target using the **`setJointValueTarget`** method.
- Finally, call the **`plan()`** method of the Move Group interface to plan the trajectory. Check whether it could be planned. Some configurations are not possible because they will collide with themselves.

Take a deeper look at the functions provided by `MoveGroupInterface` here: http://docs.ros.org/en/noetic/api/moveit_ros_planning_interface/html/classmoveit_1_1planning__interface_1_1MoveGroupInterface.html.

Create the files and launch them.

a) Inside your **ros2_ws**, create a new package named **moveit2_scripts** with dependencies.

Execute in Shell #1

cd ~/ros2_ws/src ros2 pkg create moveit2_scripts --dependencies rclcpp rclcpp_action moveit_core moveit_ros_planning moveit_ros_planning_interface interactive_markers geometric_shapes control_msgs moveit_msgs
Inside this package, create a new file named **test_trajectory.cpp** and copy the code you have seen above inside this file.

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch test_trajectory.cpp
```

b) Inside the package, also create a launch file to launch the program, called **test_trajectory.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
mkdir launch
touch launch/test_trajectory.launch.py
```

Paste the code below into this launch file:

test_trajectory.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
    moveit_config = MoveItConfigsBuilder("name",
package_name="limo_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="test_trajectory",
        package="moveit2_scripts",
        executable="test_trajectory",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )
```

```
    return LaunchDescription(
        [moveit_cpp_node]
    )
```

The launch file is pretty simple. First, load your MoveIt2 package configuration into the variable `moveit_config`:

In [ ]:

```
moveit_config = MoveItConfigsBuilder("name",
package_name="limo_moveit_config").to_moveit_configs()
```

Next, launch the `test_trajectory` executable, which will execute the `test_trajectory.cpp` program:

In [ ]:

```
moveit_cpp_node = Node(
    name="test_trajectory",
    package="moveit2_scripts",
    executable="test_trajectory",
    output="screen",
    parameters=[
        moveit_config.robot_description,
        moveit_config.robot_description_semantic,
        moveit_config.robot_description_kinematics,
        {'use_sim_time': True},
    ],
)
```

As you can see, you must pass the `robot description`, `robot semantics`, and `robot kinematics` data to your program.

c) Now, update the **CMakeLists.txt** file to compile everything. First, open the file and add the following lines to the code:

Add to CMakeLists.txt

In [ ]:

```
# Generate the executable
add_executable(test_trajectory
    src/test_trajectory.cpp)
target_include_directories(test_trajectory
    PUBLIC include)
ament_target_dependencies(test_trajectory
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
```

```
        interactive_markers
        moveit_ros_planning
        control_msgs)

# Install the executable
install(TARGETS
        test_trajectory
  DESTINATION lib/${PROJECT_NAME}
)

# Install the launch file
install(DIRECTORY
        launch
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

You are generating an executable from the file **test_trajectory.cpp** and installing it alongside the contents of the launch folder.

d) Great! Now, compile your workspace.

Execute in Shell #1

Remember that the `ros2_ws` has to be compiled to find packages inside the `/home/simulations/ros2_sims_ws`:

In [ ]:

```
cd ~/ros2_ws
rm -rf build/ install/ log/
source /home/simulations/ros2_sims_ws/install/setup.bash
colcon build
source install/setup.bash
```

e) Now, test your new program! First, launch the MoveIt2 RVIZ environment.

- Reminder -

Remember that before using the Move Group Interface, you must start the Move Group node and, optionally, MoveIt RVIZ for visualization.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws
```

```
source install/setup.bash
ros2 launch limo_moveit_config moveit_rviz.launch.py
```

- End of Reminder -

f) Now, run your code:

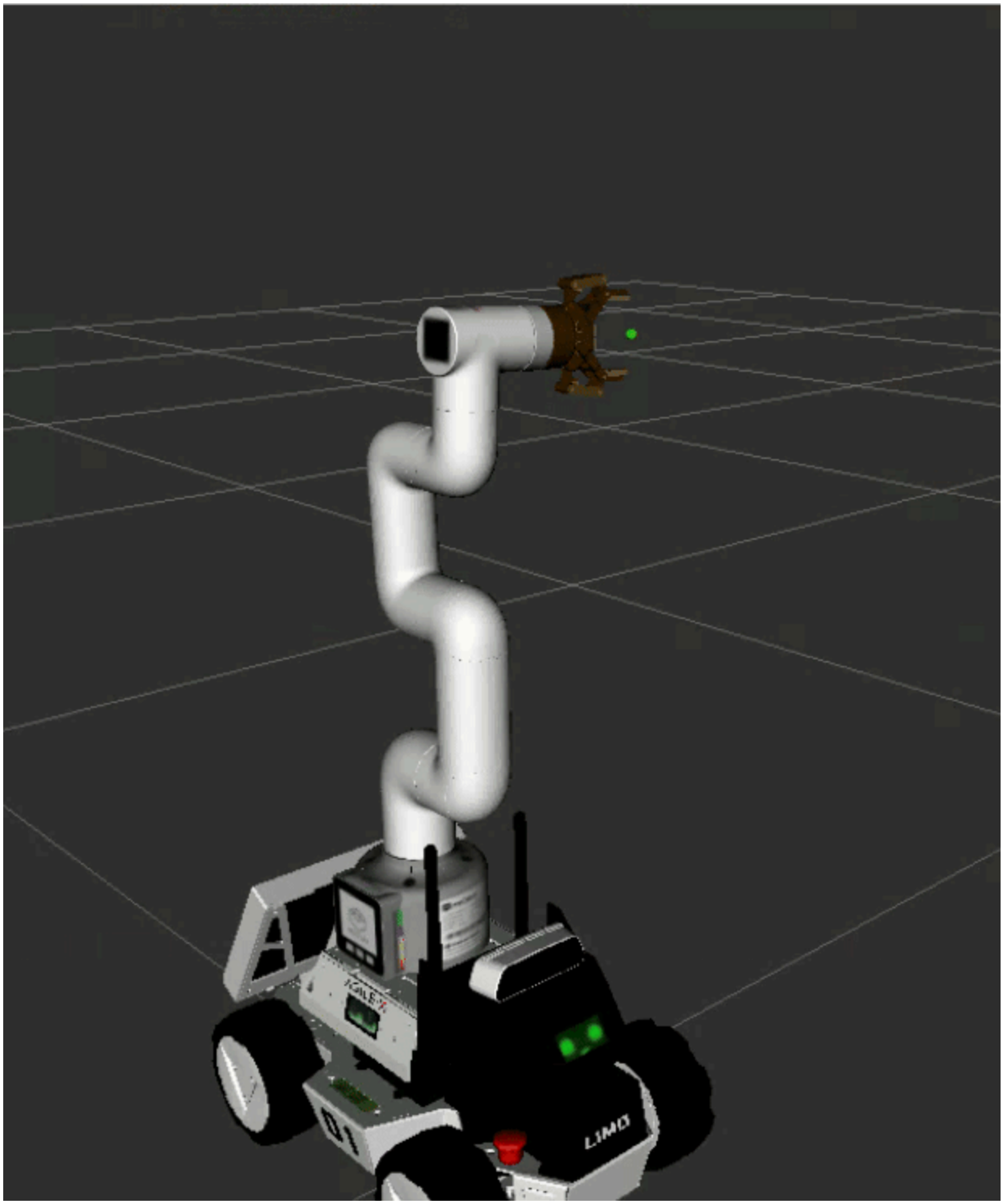
Execute in Shell #3

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch moveit2_scripts test_trajectory.launch.py
```

See how the plan to the **home** position is computed.

- Expected Result -



- End of Expected Result -

g) To plan different motions, you can modify the values assigned to the **joint_group_positions**.

- Note -

REMEMBER: You have planned a trajectory but not executed it yet, which is why the robot in the simulation is not moving. To get the robot to execute the planned trajectory. You must make the robot execute it. Learn how to do that later in this unit.

- End of Note -

2.1.2 Planning to an end effector pose

Sometimes, moving the end effector toward a desired goal pose might be more convenient. See how you can do this.

a) Create another C++ script named **test_trajectory2.cpp**

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch test_trajectory2.cpp
```

Paste the code below into this file:

test_trajectory2.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>
#include <rclcpp/rclcpp.hpp>
#include <thread>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);

  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  static const std::string PLANNING_GROUP = "ur_manipulator";
  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  // We set parameters
  move_group.setMaxVelocityScalingFactor(1.0);
  move_group.setMaxAccelerationScalingFactor(1.0);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");

  RCLCPP_INFO(LOGGER, "Planning frame: %s",
```

```
            move_group.getPlanningFrame().c_str());
  RCLCPP_INFO(LOGGER, "End effector link: %s",
            move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
          move_group.getJointModelGroupNames().end(),
          std::ostream_iterator<std::string>(std::cout, ", "));

  geometry_msgs::msg::Pose target_pose1;
  target_pose1.position.x = 0.232;
  target_pose1.position.y = -0.010;
  target_pose1.position.z = 0.334;
  // Set orientation if necessary
  target_pose1.orientation.x = 0.0;
  target_pose1.orientation.y = 0.0;
  target_pose1.orientation.z = 0.0;
  target_pose1.orientation.w = 1.0;

  move_group.setPoseTarget(target_pose1);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan;
  bool success =
      (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  if (success == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Moved to End Effector pose successfully.");
  } else {
    RCLCPP_ERROR(LOGGER, "Failed to move to End Effector pose.");
    return 1;
  }

  rclcpp::shutdown();
  return 0;
}
```

Have a closer look at the new code introduced:

In [ ]:

```
geometry_msgs::msg::Pose target_pose1;
target_pose1.position.x = 0.232;
target_pose1.position.y = -0.010;
target_pose1.position.z = 0.334;
// Set orientation if necessary
target_pose1.orientation.x = 0.0;
target_pose1.orientation.y = 0.0;
target_pose1.orientation.z = 0.0;
```

```
target_pose1.orientation.w = 1.0;

move_group.setPoseTarget(target_pose1);
```

As you can see, it is pretty simple. First, define a variable `target_pose1`. Next, assign the end effector's desired pose values (position and orientation). Finally, set this desired pose as the target for `move_group`.

b) Update the **CMakeLists.txt** file to compile the new script.

Add to CMakeLists.txt

In [ ]:

```
add_executable(test_trajectory2
   src/test_trajectory2.cpp)
target_include_directories(test_trajectory2
   PUBLIC include)
ament_target_dependencies(test_trajectory2
   ament_cmake
         rclcpp
         rclcpp_action
         moveit_core
         moveit_ros_planning_interface
         interactive_markers
         moveit_ros_planning
         control_msgs)

install(TARGETS
         test_trajectory
         test_trajectory2
   DESTINATION lib/${PROJECT_NAME}
)
```

c) Create a new launch file named **test_trajectory2.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
cp launch/test_trajectory.launch.py launch/test_trajectory2.launch.py
```

d) Modify the launch file to start the new executable:

In [ ]:

```
moveit_cpp_node = Node(
   name="test_trajectory2",
   package="moveit2_scripts",
   executable="test_trajectory2",
   output="screen",
   parameters=[
```

```
        moveit_config.robot_description,
        moveit_config.robot_description_semantic,
        moveit_config.robot_description_kinematics,
        {'use_sim_time': True},
    ],
)
```

e) Recompile your package:

Execute in Shell #3

In [ ]:

```
cd ~/ros2_ws/
source install/setup.bash
colcon build
source install/setup.bash
```

f) Finally, launch the code:

- Notes -

Remember to run MoveIt2 to run the script. Start it if you do not have it running.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config moveit_rviz.launch.py
```

- End of Notes -

Execute in Shell #3

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch moveit2_scripts test_trajectory2.launch.py
```

- Expected Result -

- End of Expected Result -

2.2 Executing a Trajectory

Now, you have seen methods that allow you to plan a trajectory with C++ code. However, what about executing this trajectory?

It is simple. Call the `execute (plan)` function from the Planning Group to execute a trajectory. For instance, given a plan `my_plan`, execute the planned motion like this:

In [ ]:

```
move_group.execute(my_plan_arm);
```

By executing this line of code, you are telling your robot to **Execute** the trajectory stored in the `my_plan` variable.
Also, there is the option to **Plan & Execute** a motion, as the GUI provides.

In [ ]:

```
move_group.move();
```

Modify the C++ programs created in the previous exercises to execute the trajectory.

Modified test_trajectory.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>

#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  // We set parameters
  move_group.setMaxVelocityScalingFactor(1.0);
  move_group.setMaxAccelerationScalingFactor(1.0);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");

  const moveit::core::JointModelGroup *joint_model_group =
      move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);
```

```
  RCLCPP_INFO(LOGGER, "Planning frame: %s",
              move_group.getPlanningFrame().c_str());

  RCLCPP_INFO(LOGGER, "End effector link: %s",
              move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
            move_group.getJointModelGroupNames().end(),
            std::ostream_iterator<std::string>(std::cout, ", "));

  moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

  std::vector<double> joint_group_positions;
  current_state->copyJointGroupPositions(joint_model_group,
                                         joint_group_positions);

  // Go Home movement
  std::vector<double> home_position = {-1.57, 0.0, 0.7, 0.0, 0.0, 0.0};
  move_group.setJointValueTarget(home_position);

  moveit::planning_interface::MoveGroupInterface::Plan my_plan;

  bool success =
      (move_group.plan(my_plan) == moveit::core::MoveItErrorCode::SUCCESS);

  if (success == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Moved to home position successfully.");
  } else {
    RCLCPP_ERROR(LOGGER, "Failed to move to home position.");
    return 1;
  }

  // Execute the trajectory
  if (move_group.execute(my_plan) ==
      moveit::planning_interface::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Moved to home position executed successfully.");
  } else {
    RCLCPP_ERROR(LOGGER, "Failed to execute home position.");
  }

  rclcpp::shutdown();
  return 0;
}
```

Recompile your package and test it:

Execute in Shell #4

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
colcon build
source install/setup.bash
```

Execute in Shell #4

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch moveit2_scripts test_trajectory.launch.py
```

- Expected Result for Exercise 2.1 -

- End of Expected Result -

- Exercise 2.1 -

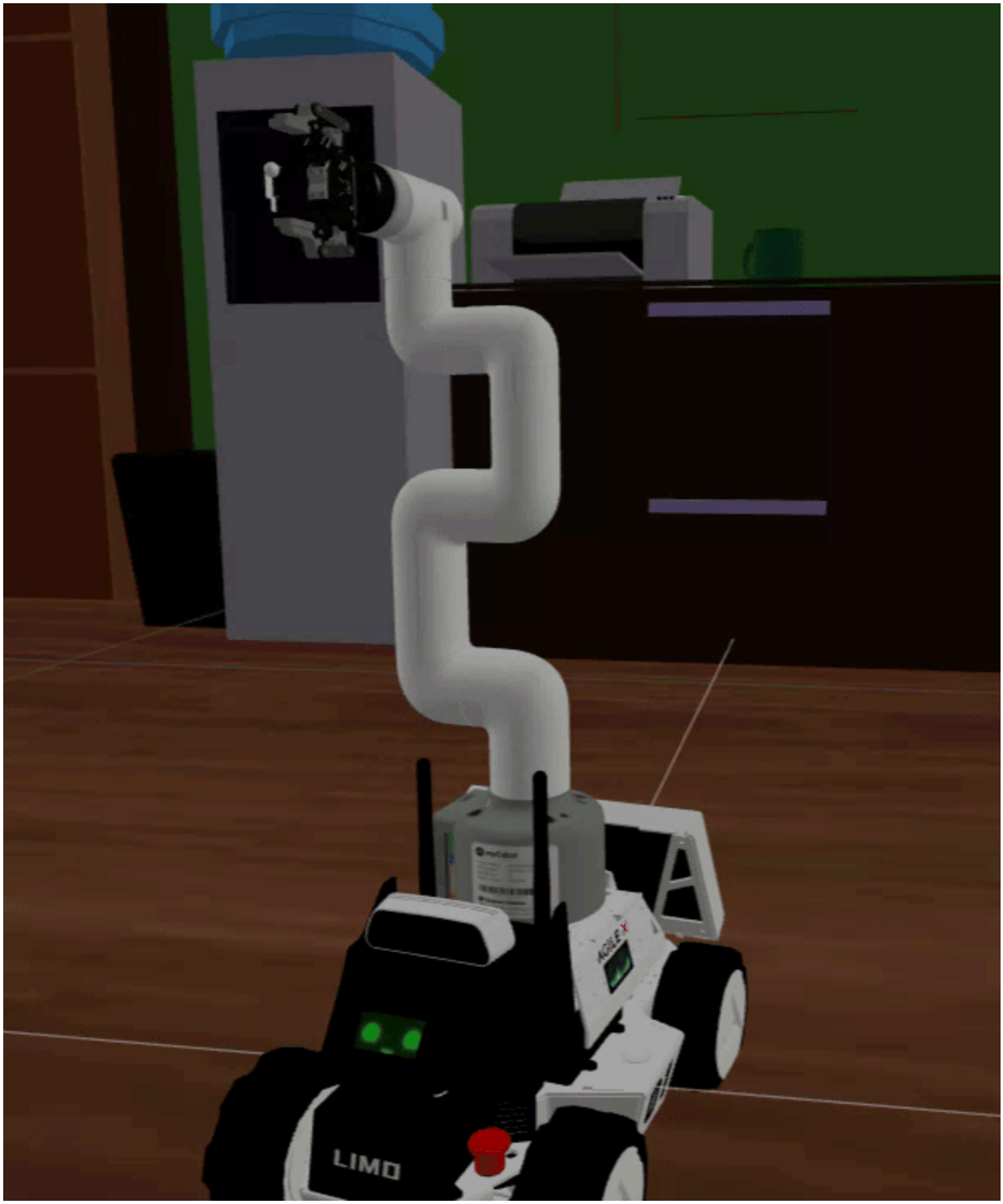
Create a new program in a C++ script named **test_trajectory3.cpp**. This new program plans and executes two different trajectories. The workflow is as follows:

- First, the program tells MoveIt2 to plan and execute a trajectory for the robotic arm to achieve a certain `pos1`.
- When the first motion finishes, the robot plans and executes the pos2.
- You have **TWO examples of positions here, but you can try others**.

pos1 = {1.57, 0.0, 0.7, 0.7, 0.0, 0.0}; pos2 = {-1.57, 0.0, 1.57, 0.0, 0.0, 0.0};

- End of Exercise 2.1 -

- Expected Result for Exercise 2.1 -



- End of Expected Result -

- Solution for Exercise 2.1 -

test_trajectory3.cpp

In [ ]:

```
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
```

```
#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP = "ur_manipulator";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  rclcpp::NodeOptions node_options;
  node_options.automatically_declare_parameters_from_overrides(true);
  auto move_group_node =
      rclcpp::Node::make_shared("move_group_interface_tutorial", node_options);

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(move_group_node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(move_group_node,
                                                            PLANNING_GROUP);

  // We set parameters
  move_group.setMaxVelocityScalingFactor(1.0);
  move_group.setMaxAccelerationScalingFactor(1.0);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");

  const moveit::core::JointModelGroup *joint_model_group =
      move_group.getCurrentState()->getJointModelGroup(PLANNING_GROUP);

  RCLCPP_INFO(LOGGER, "Planning frame: %s",
              move_group.getPlanningFrame().c_str());

  RCLCPP_INFO(LOGGER, "End effector link: %s",
              move_group.getEndEffectorLink().c_str());

  RCLCPP_INFO(LOGGER, "Available Planning Groups:");
  std::copy(move_group.getJointModelGroupNames().begin(),
            move_group.getJointModelGroupNames().end(),
            std::ostream_iterator<std::string>(std::cout, ", "));

  moveit::core::RobotStatePtr current_state = move_group.getCurrentState(10);

  std::vector<double> joint_group_positions;
  current_state->copyJointGroupPositions(joint_model_group,
                                         joint_group_positions);
```

```
  // Go To Position 1
  std::vector<double> pos1 = {1.57, 0.0, 0.7, 0.7, 0.0, 0.0};
  move_group.setJointValueTarget(pos1);

  // Execute the trajectory
  if (move_group.move() ==
      moveit::planning_interface::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Moved to home position executed successfully.");
  } else {
    RCLCPP_ERROR(LOGGER, "Failed to execute home position.");
  }

  // Go To Position 2
  std::vector<double> pos2 = {-1.57, 0.0, 1.57, 0.0, 0.0, 0.0};
  move_group.setJointValueTarget(pos2);

  // Execute the trajectory
  if (move_group.move() ==
      moveit::planning_interface::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Moved to home position executed successfully.");
  } else {
    RCLCPP_ERROR(LOGGER, "Failed to execute home position.");
  }

  rclcpp::shutdown();
  return 0;
}
```

CMakelists.txt ADD

In [ ]:

```
add_executable(test_trajectory3
    src/test_trajectory3.cpp)
target_include_directories(test_trajectory3
    PUBLIC include)
ament_target_dependencies(test_trajectory3
    ament_cmake
            rclcpp
            rclcpp_action
            moveit_core
            moveit_ros_planning_interface
            interactive_markers
            moveit_ros_planning
            control_msgs)

install(TARGETS
```

```
        test_trajectory
        test_trajectory2
        test_trajectory3
    DESTINATION lib/${PROJECT_NAME}
)
```

- End Solution for Exercise 2.1 -

2.3 How to Get Your Robot's Arms Positioned Fast and Easy

Getting the end effector pose or joints with numbers and testing is tedious and inefficient. For this purpose, make use of a tool to move the robotic arm quickly and intuitively:

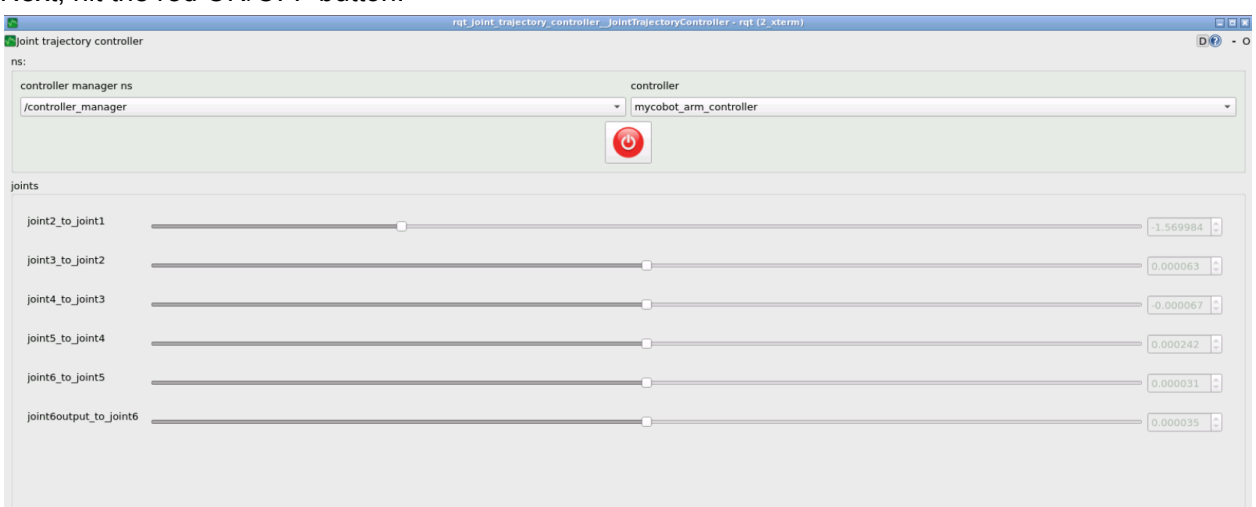
Execute in Shell #1

In [ ]:

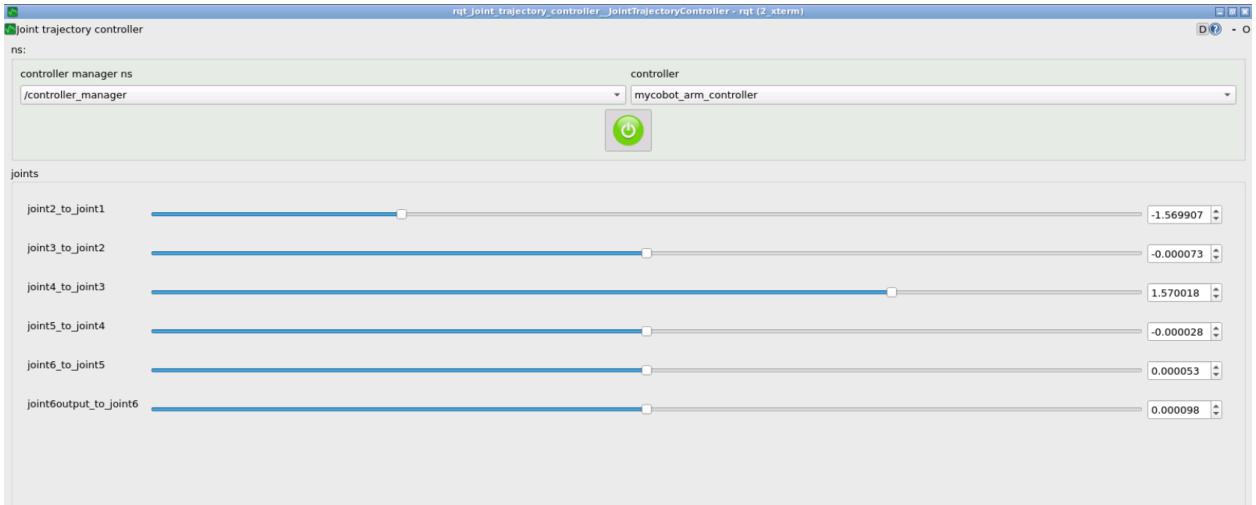
```
ros2 run rqt_joint_trajectory_controller rqt_joint_trajectory_controller
```

This command launches the `rqt_joint_trajectory_controller` node, providing a GUI to move the joints freely (remember to open the Graphical Tools screen by pressing the small screen button in the IDE).

Now, set the **controller manager ns** and **controller** variables using the options in the figure below. Next, hit the red ON/OFF button:



You should see how the button turns green and the joint slides become available.



You can now freely control each arm of the joint. Play with the slides and move the robot to different configurations.
Also, review a few methods to get the joint and end effector positions easily.

1. Get the end effector pose

You can use the command line to get the end effector's position relative to the robot base frame (or any other robot frame). For this, use the robot's **transforms** (TF).

Execute in Shell #1

In [ ]:

ros2 run tf2_ros tf2_echo base_footprint joint6_flange

Expected Output

```
At time 8345.480000000
- Translation: [0.006, -0.066, 0.560]
- Rotation: in Quaternion [0.500, -0.500, -0.500, 0.500]
- Rotation: in RPY (radian) [1.570, -0.000, -1.570]
- Rotation: in RPY (degree) [89.948, -0.002, -89.952]
- Matrix:
  0.001  0.001 -1.000  0.006
 -1.000  0.000 -0.001 -0.066
  0.000  1.000  0.001  0.560
  0.000  0.000  0.000  1.000
```

2. Get the joint positions

Additionally, you can get the current joint positions by checking the **/joint_states** topic. The **Joint State Controller** provides the joint states of the robot.

Execute in Shell #2

In [ ]:

ros2 topic echo /joint_states

Expected Output

```
Header:
  stamp:
    sec: 8411
    nanosec: 700000000
  frame_id: ''
name:
- joint2_to_joint1
- joint4_to_joint3
- joint5_to_joint4
- joint6_to_joint5
- joint3_to_joint2
- joint6output_to_joint6
- robotiq_85_left_knuckle_joint
position:
- -1.5699837875163132
- -6.662485881925306e-05
- 0.00024188591694596084
- 3.116456813501145e-05
- 6.343301344013952e-05
- 3.478360899222821e-05
- -5.594854425439166e-05
velocity:
- -0.0649958234103275
- -0.11775151450898301
- 0.2683485446515661
- 0.06669013128225593
- -0.027215833050352335
- -0.0005222672319224762
- -0.056039146752585436
effort:
- .nan
- .nan
- .nan
- .nan
- .nan
- .nan
- .nan
---
```

2.4 Controlling the Gripper

As you should have noted, the UR3e arm has a gripper mounted on its end effector. You have only been working with the robot arm. Now, get the gripper involved!

As you may have noticed, the gripper is controlled by a **GripperCommand** Action Server.

In [ ]:

/gripper_controller/gripper_cmd

To control the gripper, send a goal to its Action Server.

Execute in Shell #2

OPEN GRIPPER

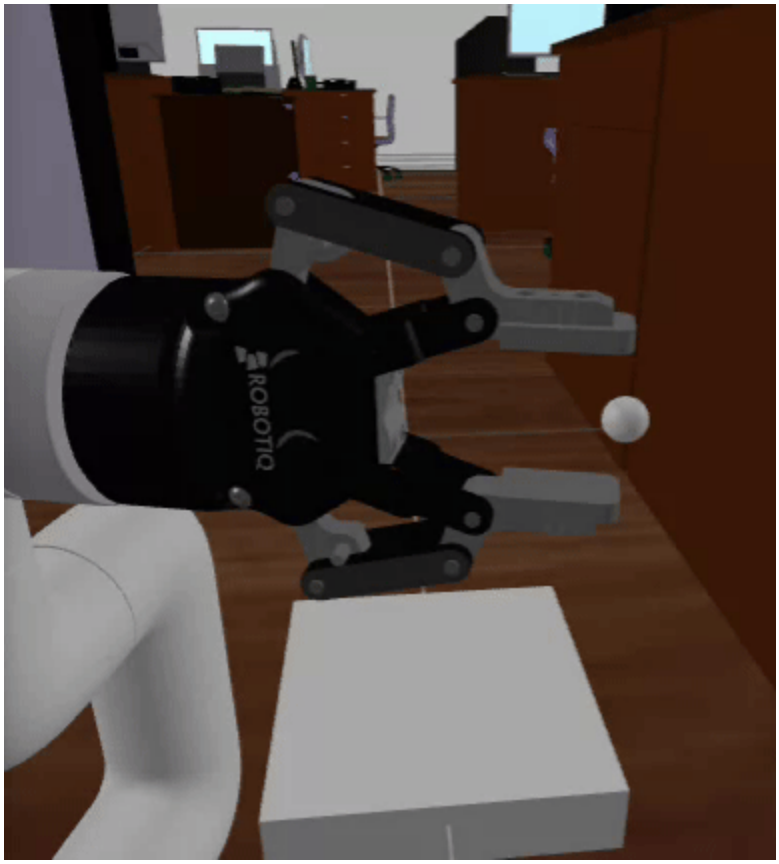
In [ ]:

```
ros2 action send_goal /gripper_controller/gripper_cmd control_msgs/action/GripperCommand
"command:
  position: 0.0
  max_effort: 0.0
"
```

CLOSE GRIPPER

In [ ]:

```
ros2 action send_goal /gripper_controller/gripper_cmd control_msgs/action/GripperCommand
"command:
  position: 0.39
  max_effort: 0.0
"
```



2.5 Controlling the Gripper with MoveIt2

You have learned how to control the arm using the C++ program. However, is it possible to do the same with the gripper? Find the answer in the next exercise!
Review the code you will execute:
test_gripper.cpp

In [ ]:

```
#include <chrono>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <moveit_msgs/msg/display_robot_state.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>
#include <rclcpp/rclcpp.hpp>
#include <thread>

static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");
static const std::string PLANNING_GROUP_GRIPPER = "gripper";

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto move_group_node = rclcpp::Node::make_shared(
      "move_group_interface_tutorial",
      rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(
```

```
        true));

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(move_group_node);
std::thread([&executor]() { executor.spin(); }).detach();

moveit::planning_interface::MoveGroupInterface move_group_gripper(
    move_group_node, PLANNING_GROUP_GRIPPER);

// Close the gripper using a named target
RCLCPP_INFO(LOGGER, "Closing the gripper.");
if (move_group_gripper.setNamedTarget("close") &&
    move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
  RCLCPP_INFO(LOGGER, "Gripper closed successfully.");
} else {
  RCLCPP_ERROR(LOGGER, "Failed to close the gripper.");
}

// Wait for a few seconds
std::this_thread::sleep_for(std::chrono::seconds(1));

// Close Custom the gripper
// Get current state of the gripper
auto current_state_gripper = move_group_gripper.getCurrentState();
std::vector<double> joint_group_positions_gripper;
current_state_gripper->copyJointGroupPositions(
    move_group_gripper.getCurrentState()->getJointModelGroup(
        PLANNING_GROUP_GRIPPER),
    joint_group_positions_gripper);

RCLCPP_INFO(LOGGER, "Closing Custom the gripper.");
joint_group_positions_gripper[2] =
    0.55; // Assuming the gripper's relevant joint is at index 2
move_group_gripper.setJointValueTarget(joint_group_positions_gripper);
if (move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
  RCLCPP_INFO(LOGGER, "Gripper Custom closed successfully.");
} else {
  RCLCPP_INFO(LOGGER, "Failed Custom closed.");
}

// Wait for a few seconds
std::this_thread::sleep_for(std::chrono::seconds(3));

// Close the gripper using a named target
RCLCPP_INFO(LOGGER, "Closing the gripper.");
if (move_group_gripper.setNamedTarget("open") &&
```

```
    move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
  RCLCPP_INFO(LOGGER, "Gripper Opened successfully.");
 } else {
  RCLCPP_ERROR(LOGGER, "Failed to Open the gripper.");
 }

 rclcpp::shutdown();
 return 0;
}
```

Take a closer look at the new code introduced!

- `Initialization`: The application begins by initializing the ROS2 system and creating a node named `move_group_interface_tutorial`. This node is configured to automatically declare parameters from overrides, facilitating the configuration of node parameters via the command line or launch files.

In [ ]:

```
auto move_group_node = rclcpp::Node::make_shared(
 "move_group_interface_tutorial",
 rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(
    true));
```

- `Executor and Threading`: A `SingleThreadedExecutor` is created, and the node is added to this executor. The executor is responsible for handling ROS2 callbacks and events. It runs in a separate thread, detached from the main thread, to continuously process incoming events.

In [ ]:

```
rclcpp::executors::SingleThreadedExecutor executor;
 executor.add_node(move_group_node);
 std::thread([&executor]() { executor.spin(); }).detach();
```

- `MoveGroupInterface Setup`: The MoveGroupInterface for the gripper is **initialized** with the node and the name of the Planning Group (`gripper`). This interface facilitates planning and executing movements with the gripper.

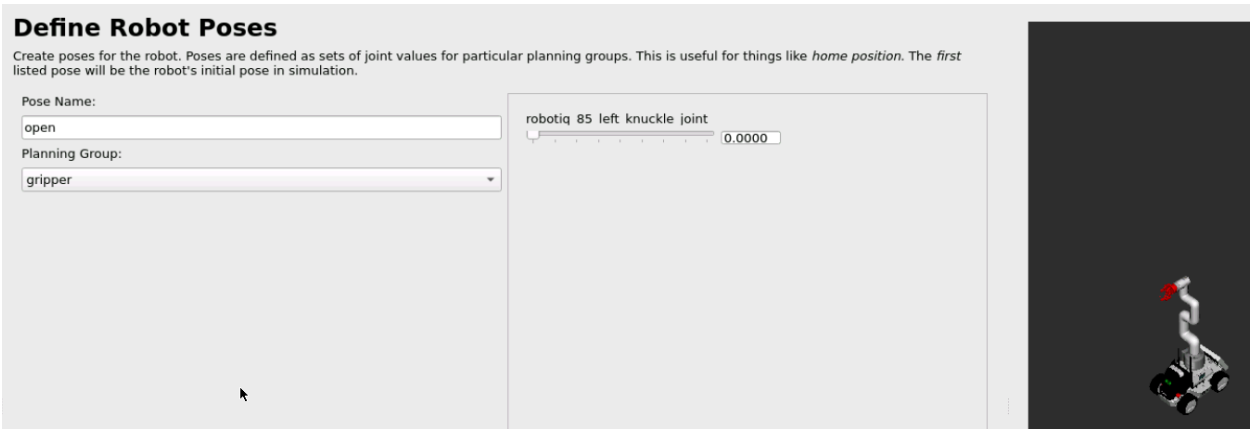
In [ ]:

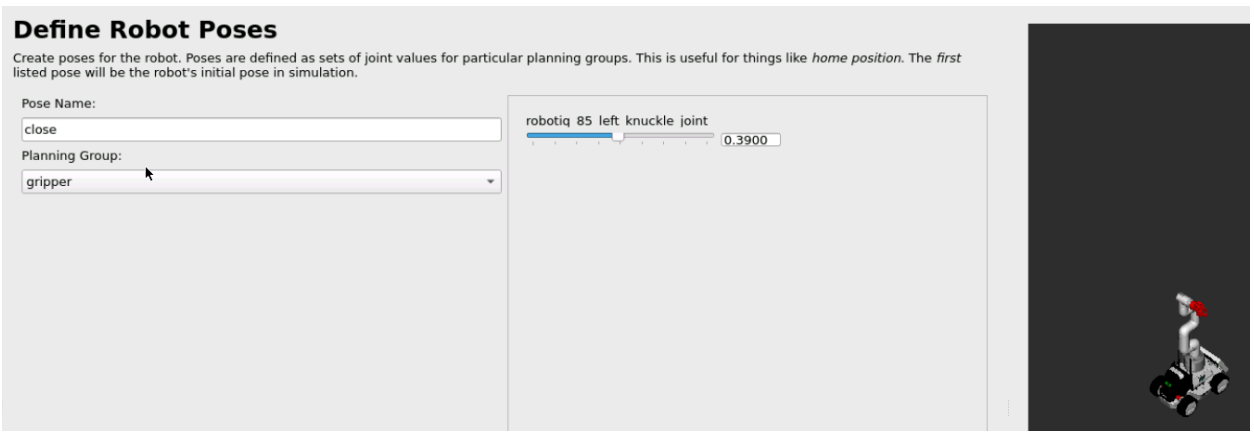
```
moveit::planning_interface::MoveGroupInterface move_group_gripper(
    move_group_node, PLANNING_GROUP_GRIPPER);
```

There are **TWO ways** to open and close the gripper:

- Through **Named Targets**: Use the movements defined in the MoveIt configuration (in this case, you defined `open` and `close`).

- Through **direct joint movement**: This method is more convoluted but allows you to programmatically close and open the gripper the amount you want. It is perfect for adapting your grip to the object to be grasped.





- **Closing the Gripper Using a Named Target**: The code first attempts to close the gripper by setting a named target "close" and executing the move. Wait **1 second** to avoid executing the next movement too fast.

In [ ]:

```
// Close the gripper using a named target
 RCLCPP_INFO(LOGGER, "Closing the gripper.");
 if (move_group_gripper.setNamedTarget("close") &&
    move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
  RCLCPP_INFO(LOGGER, "Gripper closed successfully.");
 } else {
  RCLCPP_ERROR(LOGGER, "Failed to close the gripper.");
 }

 // Wait for a few seconds
 std::this_thread::sleep_for(std::chrono::seconds(1));
```

- **Custom Close Operation**: The application retrieves the current state of the gripper, including joint positions, and manually sets the position of a specific joint (2) to `0.55` to close

the gripper. This demonstrates how to manipulate individual joint positions for custom movements.

In [ ]:

```
// Close Custom the gripper
  // Get current state of the gripper
  auto current_state_gripper = move_group_gripper.getCurrentState();
  std::vector<double> joint_group_positions_gripper;
  current_state_gripper->copyJointGroupPositions(
      move_group_gripper.getCurrentState()->getJointModelGroup(
          PLANNING_GROUP_GRIPPER),
      joint_group_positions_gripper);

  RCLCPP_INFO(LOGGER, "Closing Custom the gripper.");
  joint_group_positions_gripper[2] =
      0.55; // Assuming the gripper's relevant joint is at index 2
  move_group_gripper.setJointValueTarget(joint_group_positions_gripper);
  if (move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
    RCLCPP_INFO(LOGGER, "Gripper Custom closed successfully.");
  } else {
    RCLCPP_INFO(LOGGER, "Failed Custom closed.");
  }

  // Wait for a few seconds
  std::this_thread::sleep_for(std::chrono::seconds(3));
```

- **Opening the Gripper Using a Named Target**: After a pause, the code attempts to open the gripper using another named target, "open". Like the close operation, it logs the outcome.

In [ ]:

```
// Close the gripper using a named target
RCLCPP_INFO(LOGGER, "Closing the gripper.");
if (move_group_gripper.setNamedTarget("open") &&
  move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS) {
RCLCPP_INFO(LOGGER, "Gripper Opened successfully.");
} else {
RCLCPP_ERROR(LOGGER, "Failed to Open the gripper.");
}
```

Create the files and launch them.

a) Inside your **ros2_ws**, create a new package named **moveit2_scripts** with dependencies.
b) Inside the **moveit2_scripts** package, create a new file named **test_gripper.cpp** and copy the code into it.

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
```

touch test_gripper.cpp

c) Create a launch file inside the package to launch the program called **test_gripper.launch.py**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts
touch launch/test_gripper.launch.py
```

Paste the code below into this launch file:

test_gripper.launch.py

In [ ]:

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from moveit_configs_utils import MoveItConfigsBuilder


def generate_launch_description():
    moveit_config = MoveItConfigsBuilder("name",
package_name="my_moveit_config").to_moveit_configs()

    # MoveItCpp demo executable
    moveit_cpp_node = Node(
        name="test_gripper",
        package="moveit2_scripts",
        executable="test_gripper",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {'use_sim_time': True},
        ],
    )

    return LaunchDescription(
        [moveit_cpp_node]
    )
```

d) Now, update the **CMakeLists.txt** file to compile everything. First, open the file and add the following lines to the code:

Add to CMakeLists.txt

In [ ]:

```
add_executable(test_gripper
   src/test_gripper.cpp)
target_include_directories(test_gripper
   PUBLIC include)
ament_target_dependencies(test_gripper
   ament_cmake
         rclcpp
         rclcpp_action
         moveit_core
         moveit_ros_planning_interface
         interactive_markers
         moveit_ros_planning
         control_msgs)

# Install the executables
install(TARGETS
         test_trajectory
         test_trajectory2
         test_trajectory3
         test_gripper
   DESTINATION lib/${PROJECT_NAME}
)


# Install the launch file
install(DIRECTORY
         launch
   DESTINATION share/${PROJECT_NAME}
)
```

e) Great! Now, compile your workspace.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
rm -rf build/ install/ log/
source /home/simulations/ros2_sims_ws/install/setup.bash
colcon build
source install/setup.bash
```

f) Now, test your new program! First, launch the MoveIt2 RVIZ environment.

- Reminder -

Remember that before using the Move Group interface, you must start the Move Group node and, optionally, MoveIt RVIZ for visualization.

Execute in Shell #1

In [ ]:

ros2 launch my_moveit_config move_group.launch.py

Execute in Shell #2

In [ ]:

ros2 launch my_moveit_config moveit_rviz.launch.py

- End of Reminder -

g) Now, run your code:

Execute in Shell #3

In [ ]:

ros2 launch moveit2_scripts test_gripper.launch.py

You should now see the gripper close and open.

- Expected Result -

- End of Expected Result -

2.6 Approach & Retreat

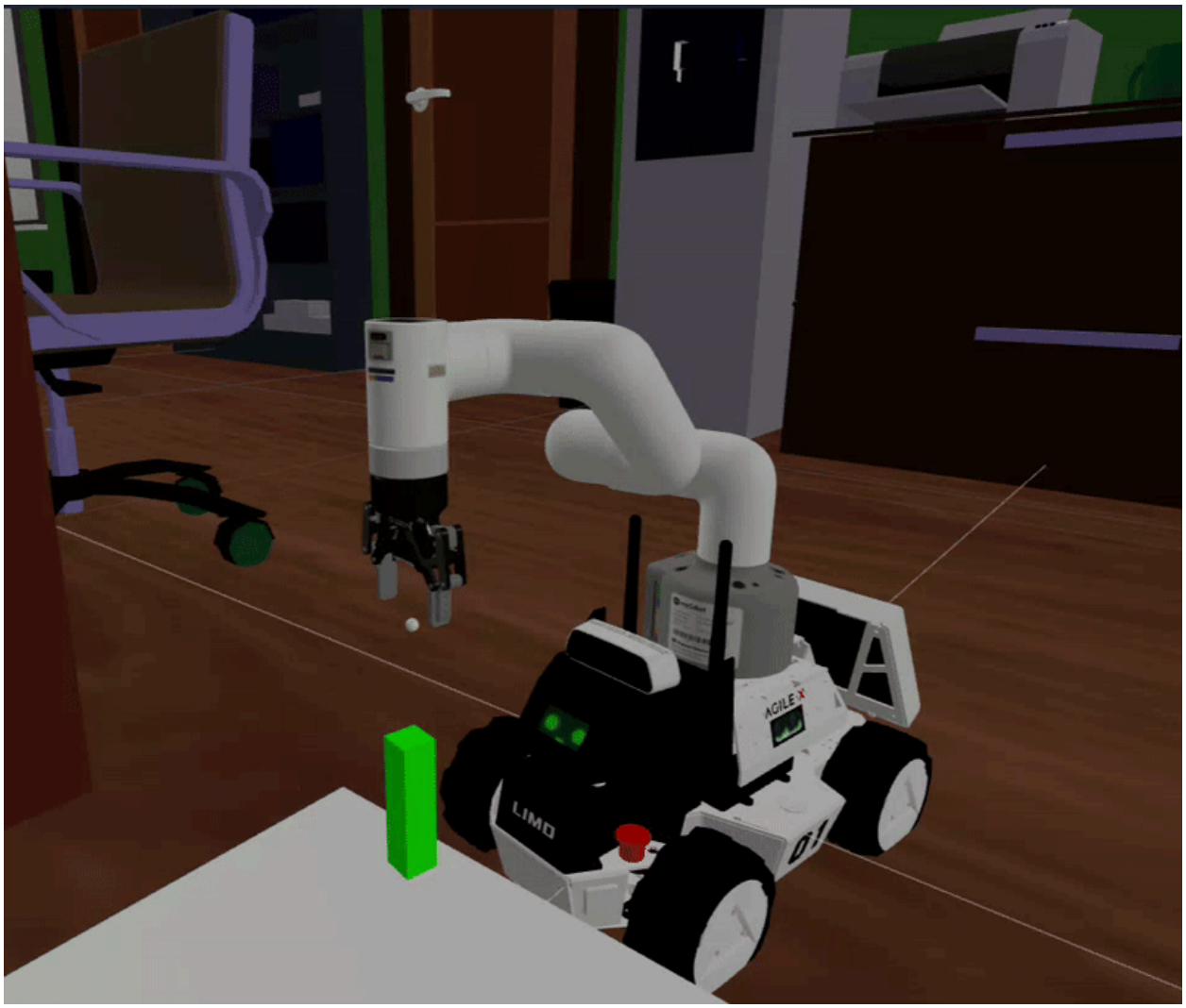
You have learned how to control the robotic arm and the gripper. Now, try to put everything together! For that, complete an exercise that performs the `approach & retreat` motions.

In manipulation, when you want to pick up an object in the environment, you do not send the end effector directly to the pose of the object because the gripper may collide with it. Instead, send the end effector to a pose **near** the object. Execute the **approach** motion to get close enough to pick up the object. Once you have picked up the object, execute the **retreat** motion to return to the previous position.

The entire sequence would look something like this:

- Exercise 2.2 -

1. Move the arm to a home position to safely start grasping, perception, etc.



2. Move the arm to a pose near the object, pregrasp, a safe position to start the grasping approach:

3. Approach:

3. Grasp:

3. Retreat:

- Note that in the approach movement, only the Z-axis position of the end effector is changed.
- The trajectory it follows is a **straight line**.
- This is known as a **Cartesian Path**.
- **Cartesian Paths** are generated by defining waypoints where your end effector should go, following a straight line, and thus better controlling its trajectory.
- Here, you have the code to perform Cartesian Paths:

In [ ]:

```
bool performCartesianPath(
   moveit::planning_interface::MoveGroupInterface &move_group,
   const std::vector<geometry_msgs::msg::Pose> &waypoints,
   double jump_threshold, double eef_step,
   moveit_msgs::msg::RobotTrajectory &trajectory) {
 double fraction = move_group.computeCartesianPath(waypoints, eef_step,
                                        jump_threshold, trajectory);
 if (fraction > 0.0) {
   return move_group.execute(trajectory) ==
         moveit::planning_interface::MoveItErrorCode::SUCCESS;
```

```
  }
  return false;
}
```

- `move_group`: The MoveGroupInterface object provides a high-level interface to move a group of joints. It is used to plan and execute motions.
- `waypoints`: A list of poses (positions and orientations) that the end effector should follow. Each pose in this vector represents a point along the path the robot arm should move through.
- `jump_threshold`: A parameter to prevent the robot from making large jumps in joint space between waypoints. A jump is considered too large if the joint's change in position exceeds this threshold.
- `eef_step`: The maximum distance in meters the end effector should move between consecutive points in the generated path. This helps in generating a smooth Cartesian Path.
- `trajectory`: A reference to a robot trajectory message where the planned path will be stored.

You can use it like this:

In [ ]:

```
// Approach movement
std::vector<geometry_msgs::msg::Pose> waypoints = {target_pose};
target_pose.position.z -= 0.05;   // Approach delta
// We can add as many waypointa as we want
waypoints.push_back(target_pose); // Add adjusted pose to waypoints

moveit_msgs::msg::RobotTrajectory trajectory;
if (!performCartesianPath(move_group, waypoints, 0.0, 0.01, trajectory)) {
   RCLCPP_ERROR(LOGGER, "Failed to execute approach.");
return 1; // Exit if the movement failed
}
RCLCPP_INFO(LOGGER, "Approach executed successfully.");
```

Alright! Try to implement the **approach & retreat** sequence in the following exercise.

- Exercise 2.2 -

In [0]:

```
* Create a script that performs the following movements:
```

a) Before getting started, spawn an object into the scene. For that, create a new file named `grasp_box.urdf` inside `~/ros2_ws/src`:

Execute in Shell

In [ ]:

```
cd ~/ros2_ws/src
touch grasp_box.urdf
```

grasp_box.urdf

In [ ]:

```
<robot name="grasp_box">

   <!-- Colours for RVIZ for geometric elements -->
   <material name="blue">
      <color rgba="0 0 0.8 1"/>
   </material>
   <material name="red">
      <color rgba="0.8 0 0 1"/>
   </material>
   <material name="green">
      <color rgba="0 0.8 0 1"/>
   </material>
   <material name="grey">
      <color rgba="0.75 0.75 0.75 1"/>
   </material>
   <material name="white">
      <color rgba="1.0 1.0 1.0 1"/>
   </material>
   <material name="black">
      <color rgba="0 0 0 1"/>
   </material>


         <!-- * * * Link Definitions * * * -->

   <link name="grasp_box_base_link">
            <inertial>
         <origin xyz="0 0 0" rpy="0 0 0"/>
         <mass value="0.01" />
         <inertia ixx="5.666666666666668e-06" ixy="0.0" ixz="0.0" iyy="5.666666666666668e-06"
iyz="0.0" izz="6.666666666666667e-07"/>
      </inertial>
      <collision>
         <origin xyz="0 0 0" rpy="0 0 0"/>
         <geometry>
            <box size="0.02 0.02 0.08"/>
         </geometry>
      </collision>
      <visual>
         <origin rpy="0.0 0 0" xyz="0 0 0"/>
         <geometry>
            <box size="0.02 0.02 0.08"/>
         </geometry>
         <material name="green"/>
```

```
    </visual>
  </link>

  <gazebo reference="grasp_box_base_link">
    <kp>100000.0</kp>
    <kd>100.0</kd>
    <mu1>0.25</mu1>
    <mu2>0.25</mu2>
    <material>Gazebo/Green</material>
  </gazebo>

</robot>
```

- Download a ROS2 package that allows you to spawn objects in Gazebo.
- A version of this script exists in the gazebo_ros package. Unfortunately, when we created this course, an error prevented it from working. For that reason, we provide a stable script that will work.

Execute in Shell 1

In [ ]:

```
cd ~/ros2_ws/src
git clone https://bitbucket.org/theconstructcore/gazebo_tools.git
cd ~/ros2_ws
source install/setup.bash
colcon build --packages-select gazebo_spawn_tools
source install/setup.bash
```

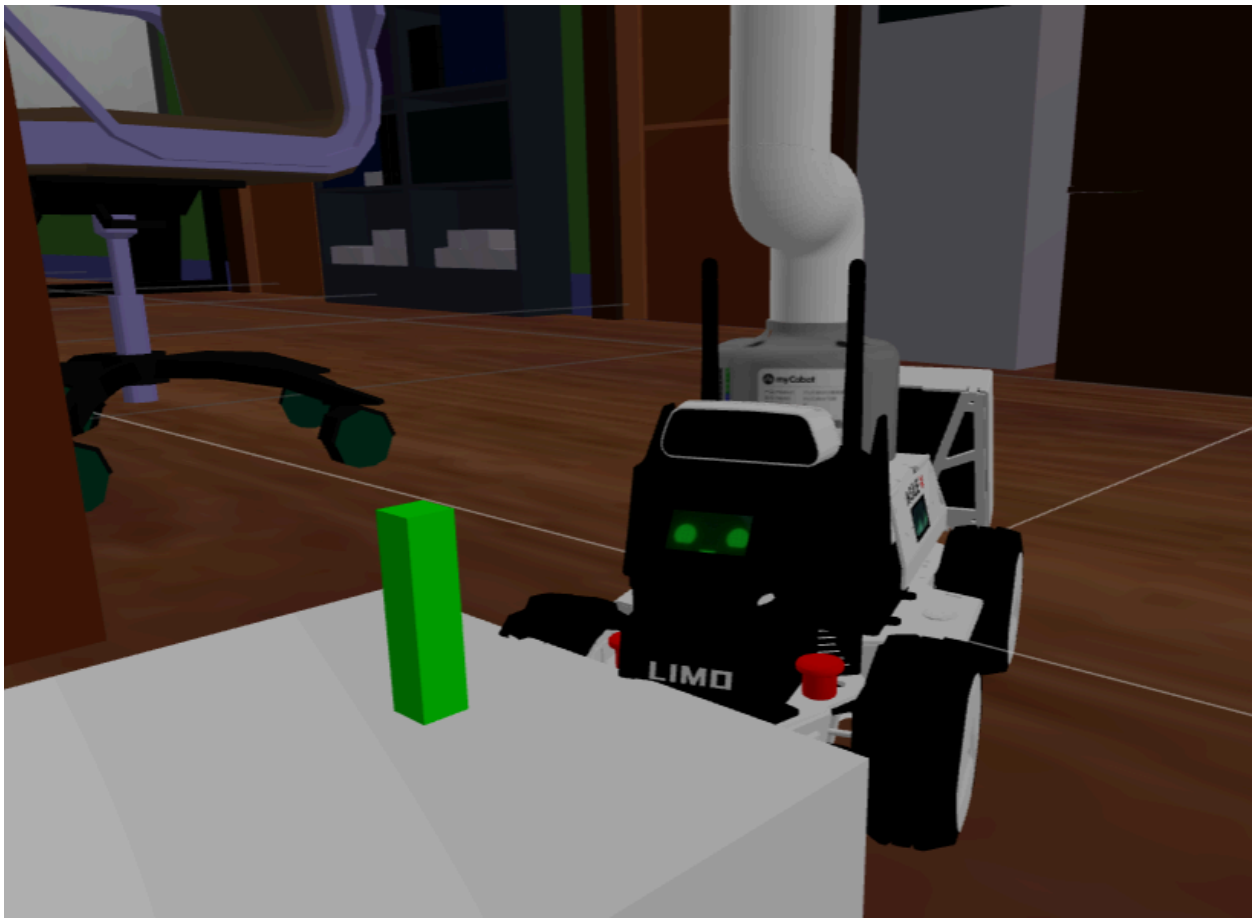
Now, execute the following command to spawn the object into the simulation.

Execute in Shell #1

In [ ]:

```
ros2 run gazebo_spawn_tools spawn_entity.py -file ~/ros2_ws/src/grasp_box.urdf -x 0.257792 -y 0
-z 0.113 -entity grasp_box
```

You will see something like this:



In your **moveit2_scripts** package, create a new file named **approach_retreat.cpp**.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/src
touch approach_retreat.cpp
```

The program generated should perform the following tasks:

- Go to the **Home** position.
- Go to a pose near the object. For this, you can use the following pose:

In [ ]:

```
geometry_msgs::msg::Pose target_pose;
// Example target pose for pregrasp position
target_pose.position.x = 0.232;
target_pose.position.y = -0.010;
target_pose.position.z = 0.334;
target_pose.orientation.w = 1.0;
```

- Open the gripper.
- Approach the object. Use a `delta` variable to move the end effector down **five centimeters**.

In [ ]:

```
float delta = 0.05;
```

In [ ]:

```
target_pose.position.z = target_pose.position.z - delta;
```

- Close the gripper to pick up the object.
- Retreat. Use a `delta` variable to move the end effector up **five centimeters**.

In [ ]:

```
target_pose.position.z = target_pose.position.z + delta;
```

- Open the gripper and release the object.

a) Create a launch file named **approach_retreat.launch.py** to start your program.

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws/src/moveit2_scripts/launch
touch approach_retreat.launch.py
```

b) Run the following commands to complete the approach & retreat sequence:

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config moveit_rviz.launch.py
```

Execute in Shell 3

- To allow correct grasping in Gazebo, start a tiny node that manages that; otherwise, Gazebo's physics cannot do the job correctly.
- We give the names of the following:
  - The link to connect the object to. In this case, the `joint6_flange`.
  - The name of the object. In this case, `graspbox`.
  - And the distance from the `joint6_falnge` link to set the center of your object.
- This service is called when the gripper closes/opens.

In [ ]:

```
cd ~/ros2_ws
```

```
source install/setup.bash
ros2 run attach_gazebo attach_client_v2_node joint6_flange grasp_box -0.12
```

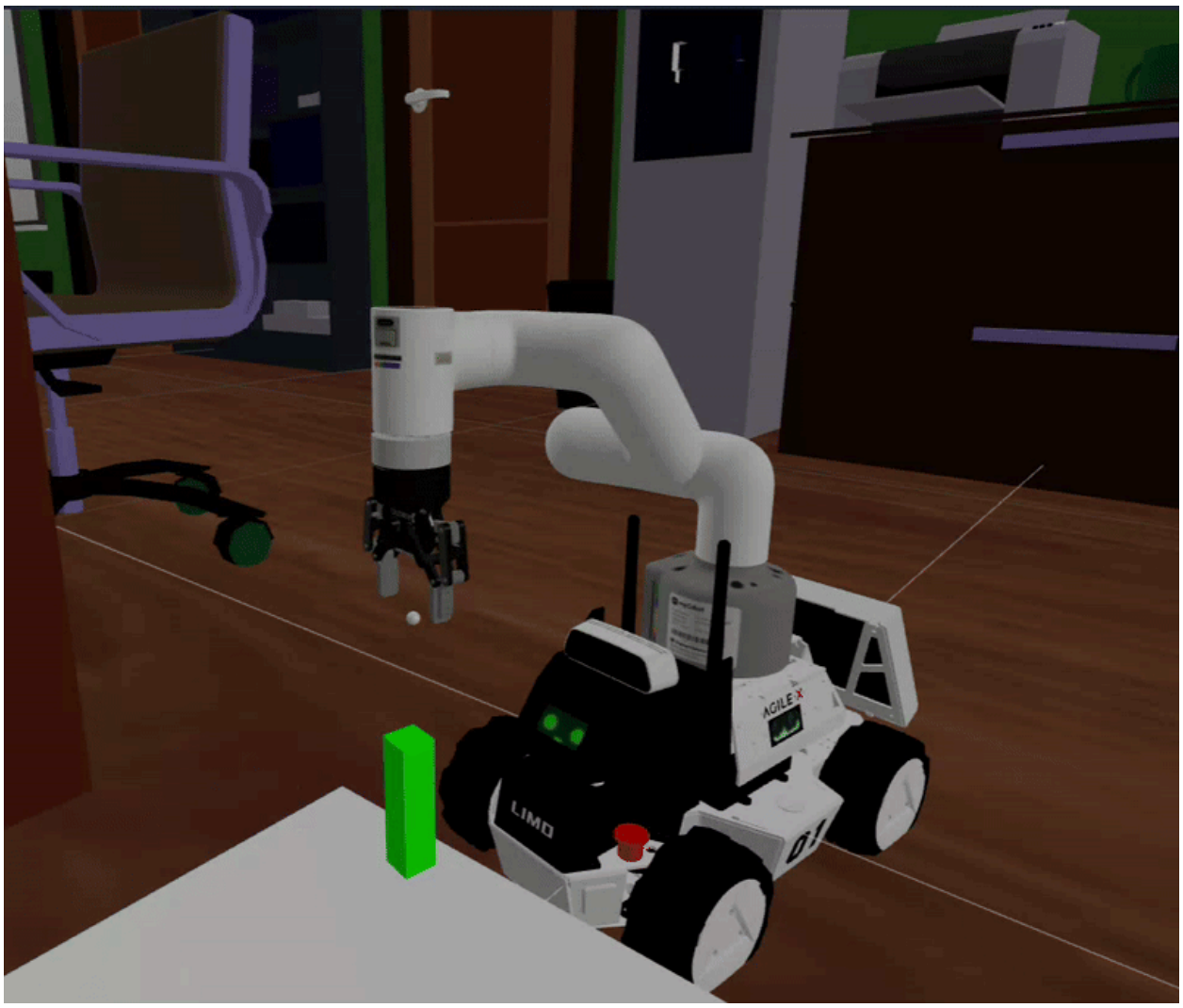
Execute in Shell 4

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch moveit2_scripts approach_retreat.launch.py
```

- Expected Result for Exercise 2.2 -

IMPORTANT NOTE: The approach & retreat motions might look different for you, but they will not resemble the GIFs shown in this section's introduction. Keep reading to understand why.



- End of Expected Result -

- End of Exercise 2.2 -

- Solution for Exercise 2.2 -

approach_retreat.cpp

In [ ]:

```
#include <geometry_msgs/msg/pose.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <rclcpp/rclcpp.hpp>
#include <string>
#include <vector>

// Logger for easy logging
static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

// Planning group names
static const std::string PLANNING_GROUP = "ur_manipulator";
static const std::string PLANNING_GROUP_GRIPPER = "gripper";

// Function to configure MoveGroup parameters
void configureMoveGroup(
    moveit::planning_interface::MoveGroupInterface &move_group) {
  move_group.setMaxVelocityScalingFactor(0.5);
  move_group.setMaxAccelerationScalingFactor(0.5);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");
}

// Function to perform movement to a home position
bool moveToHomePosition(
    moveit::planning_interface::MoveGroupInterface &move_group) {
  std::vector<double> home_position = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
  move_group.setJointValueTarget(home_position);
  return move_group.move() ==
         moveit::planning_interface::MoveItErrorCode::SUCCESS;
}

// Function to move to a specified pose target
bool moveToPoseTarget(
    moveit::planning_interface::MoveGroupInterface &move_group,
    const geometry_msgs::msg::Pose &target_pose) {
  move_group.setPoseTarget(target_pose);
  return move_group.move() ==
         moveit::planning_interface::MoveItErrorCode::SUCCESS;
}

// Function to control the gripper (open or close)
bool controlGripper(
    moveit::planning_interface::MoveGroupInterface &move_group_gripper,
    const std::string &action) {
```

```
  move_group_gripper.setNamedTarget(action);
  return move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS;
}

// Function to perform Cartesian path movement
bool performCartesianPath(
    moveit::planning_interface::MoveGroupInterface &move_group,
    const std::vector<geometry_msgs::msg::Pose> &waypoints,
    double jump_threshold, double eef_step,
    moveit_msgs::msg::RobotTrajectory &trajectory) {
  double fraction = move_group.computeCartesianPath(waypoints, eef_step,
                                        jump_threshold, trajectory);
  if (fraction > 0.0) {
    return move_group.execute(trajectory) ==
           moveit::planning_interface::MoveItErrorCode::SUCCESS;
  }
  return false;
}

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto node = rclcpp::Node::make_shared("move_group_demo");
  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(node,
                                                PLANNING_GROUP);
  moveit::planning_interface::MoveGroupInterface move_group_gripper(
      node, PLANNING_GROUP_GRIPPER);

  configureMoveGroup(move_group);

  // GO HOME
  if (!moveToHomePosition(move_group)) {
    RCLCPP_ERROR(LOGGER, "Failed to move to home position.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Moved to home position successfully.");

  geometry_msgs::msg::Pose target_pose;
  // Example target pose for pregrasp position
  target_pose.position.x = 0.232;
  target_pose.position.y = -0.010;
  target_pose.position.z = 0.334;
  target_pose.orientation.w = 1.0;
```

```
  if (!moveToPoseTarget(move_group, target_pose)) {
    RCLCPP_ERROR(LOGGER, "Failed to move to pregrasp position.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Moved to pregrasp position successfully.");

  // Approach movement
  std::vector<geometry_msgs::msg::Pose> waypoints = {target_pose};
  target_pose.position.z -= 0.05;   // Approach delta
  waypoints.push_back(target_pose); // Add adjusted pose to waypoints

  moveit_msgs::msg::RobotTrajectory trajectory;
  if (!performCartesianPath(move_group, waypoints, 0.0, 0.01, trajectory)) {
    RCLCPP_ERROR(LOGGER, "Failed to execute approach.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Approach executed successfully.");

  // Gripper control
  std::string gripper_action = "close";
  if (!controlGripper(move_group_gripper, gripper_action)) {
    RCLCPP_ERROR(LOGGER, "Failed to %s gripper!", gripper_action.c_str());
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "%s Gripper successfully.",
              gripper_action == "open" ? "Opened" : "Closed");

  // Simplified retreat movement by reusing moveToPoseTarget
  // Adjust the target pose for retreat
  target_pose.position.z += 0.05; // Retreat delta
  if (!moveToPoseTarget(move_group, target_pose)) {
    RCLCPP_ERROR(LOGGER, "Failed to retreat.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Retreat executed successfully.");

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 2.2 -

I know what you are asking yourself. Why is the arm performing these complex motions when I just want it to move a little? The answer is IK. Since you are setting a new end effector pose, new IK solutions are computed, which result in overly complex motions.

The next question you are asking yourself is: Can this behavior be changed? The answer is YES! However, you will need to continue to the next unit!

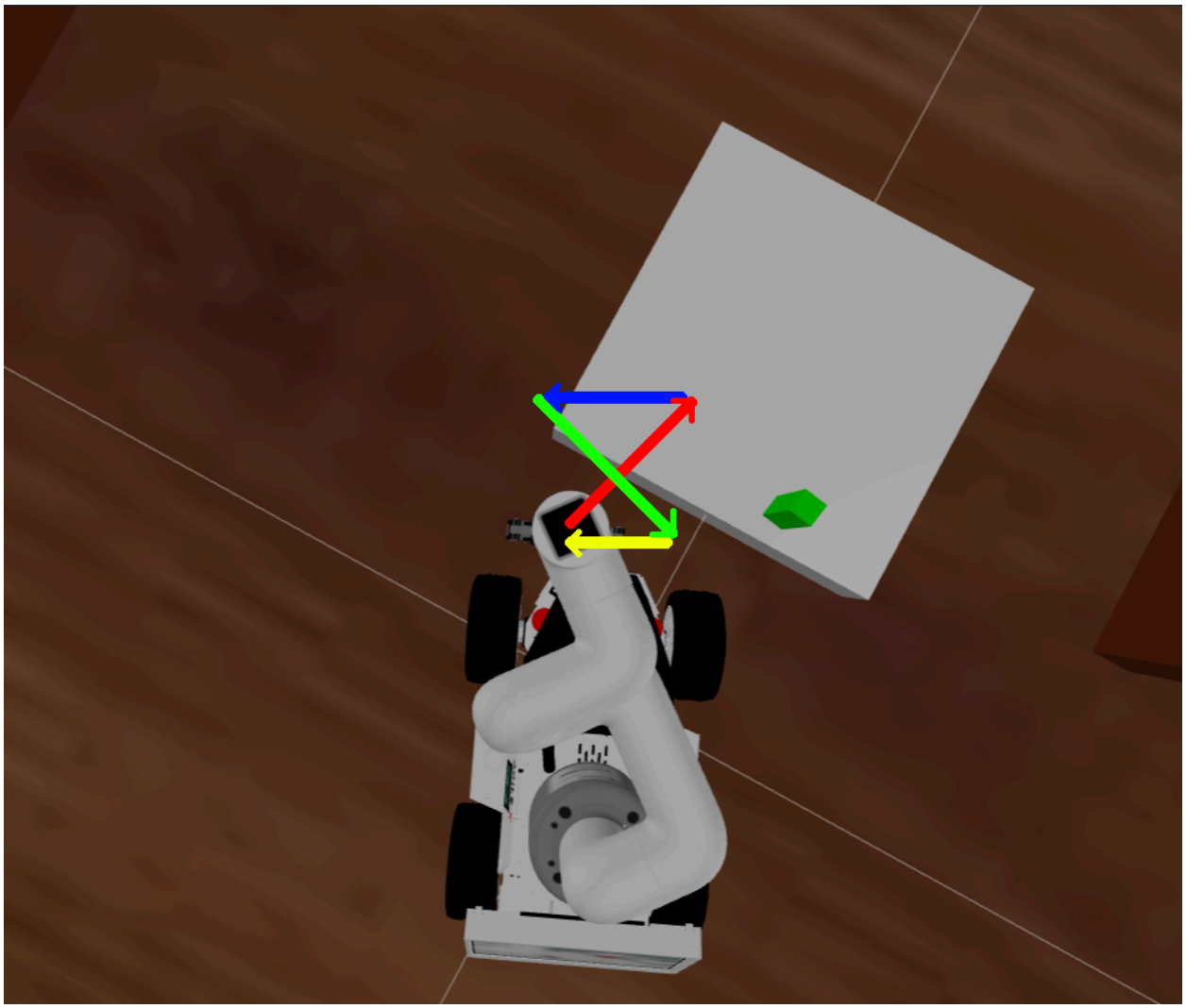
2.7 Cartesian Paths

Here is an additional exercise to understand the features and utility of using Cartesian Paths more effectively, beyond the approach & retreat maneuvers you performed, which were trivial trajectories.

- Exercise 2.3 -

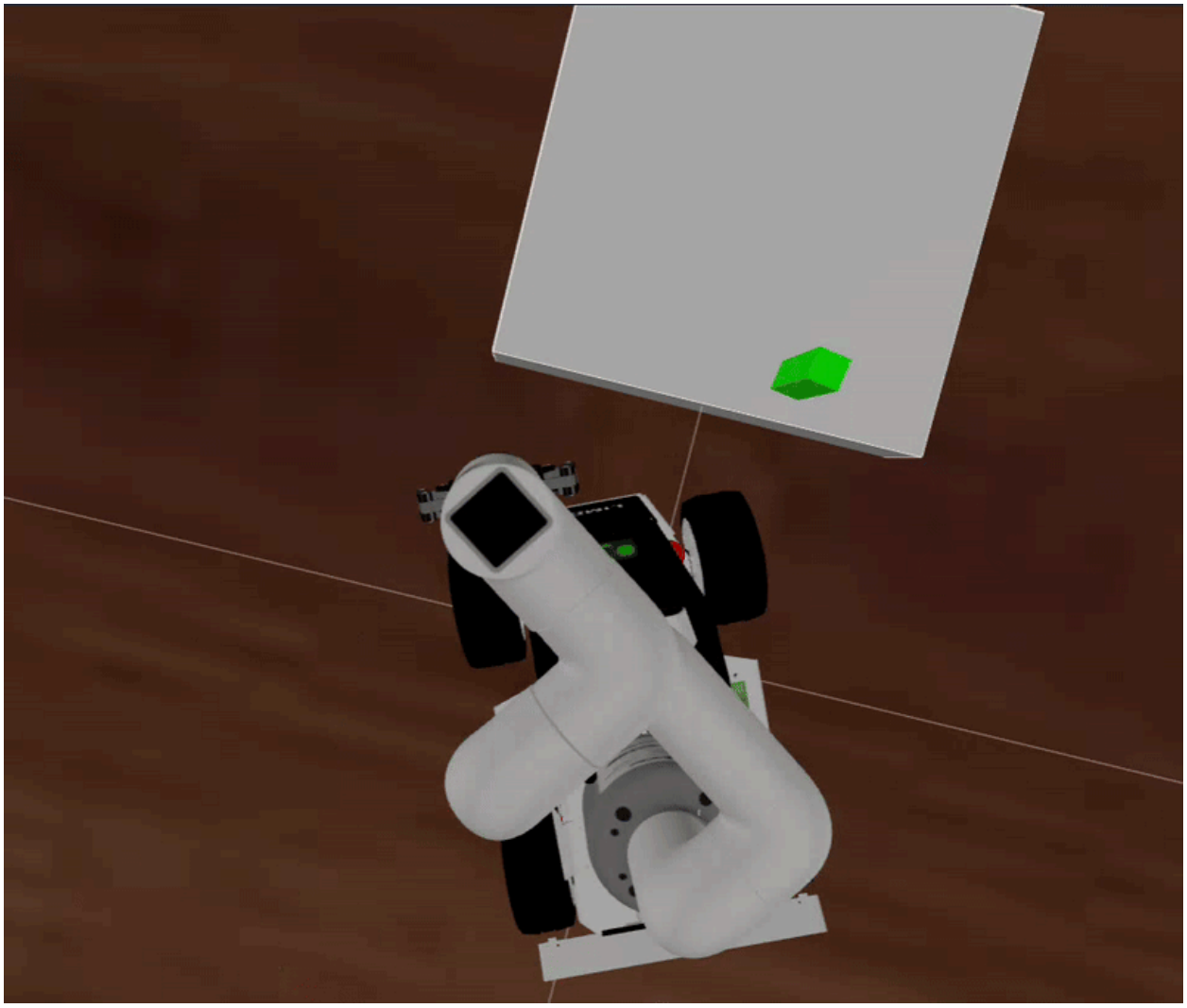
a) Create a new C++ program named **`draw_x.cpp`**. Inside this program, use `approach_retreat.cpp` as the base.

b) Modify the code to "draw" an X in the air with the robot's end effector.
Expected motion:



- End of Exercise 4.2 -

- Expected Result for Exercise 4.2 -



- End of Expected Result -

- Solution for Exercise 4.2 -

draw_x.cpp

In [ ]:

```
#include <geometry_msgs/msg/pose.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <rclcpp/rclcpp.hpp>
#include <string>
#include <vector>

// Logger for easy logging
static const rclcpp::Logger LOGGER = rclcpp::get_logger("move_group_demo");

// Planning group names
static const std::string PLANNING_GROUP = "ur_manipulator";
static const std::string PLANNING_GROUP_GRIPPER = "gripper";
```

```
// Function to configure MoveGroup parameters
void configureMoveGroup(
    moveit::planning_interface::MoveGroupInterface &move_group) {
  move_group.setMaxVelocityScalingFactor(1.0);
  move_group.setMaxAccelerationScalingFactor(1.0);
  move_group.setPlanningTime(10.0);
  move_group.setPoseReferenceFrame("base_footprint");
}

// Function to perform movement to a home position
bool moveToHomePosition(
    moveit::planning_interface::MoveGroupInterface &move_group) {
  std::vector<double> home_position = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
  move_group.setJointValueTarget(home_position);
  return move_group.move() ==
         moveit::planning_interface::MoveItErrorCode::SUCCESS;
}

// Function to move to a specified pose target
bool moveToPoseTarget(
    moveit::planning_interface::MoveGroupInterface &move_group,
    const geometry_msgs::msg::Pose &target_pose) {
  move_group.setPoseTarget(target_pose);
  return move_group.move() ==
         moveit::planning_interface::MoveItErrorCode::SUCCESS;
}

// Function to control the gripper (open or close)
bool controlGripper(
    moveit::planning_interface::MoveGroupInterface &move_group_gripper,
    const std::string &action) {
  move_group_gripper.setNamedTarget(action);
  return move_group_gripper.move() == moveit::core::MoveItErrorCode::SUCCESS;
}

// Function to perform Cartesian path movement
bool performCartesianPath(
    moveit::planning_interface::MoveGroupInterface &move_group,
    const std::vector<geometry_msgs::msg::Pose> &waypoints,
    double jump_threshold, double eef_step,
    moveit_msgs::msg::RobotTrajectory &trajectory) {
  double fraction = move_group.computeCartesianPath(waypoints, eef_step,
                                                    jump_threshold, trajectory);
  if (fraction > 0.0) {
    return move_group.execute(trajectory) ==
```

```
        moveit::planning_interface::MoveItErrorCode::SUCCESS;
  }
  return false;
}

int main(int argc, char **argv) {
  rclcpp::init(argc, argv);
  auto node = rclcpp::Node::make_shared("move_group_demo");
  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(node);
  std::thread([&executor]() { executor.spin(); }).detach();

  moveit::planning_interface::MoveGroupInterface move_group(node,
                                                 PLANNING_GROUP);
  moveit::planning_interface::MoveGroupInterface move_group_gripper(
      node, PLANNING_GROUP_GRIPPER);

  configureMoveGroup(move_group);

  // GO HOME
  if (!moveToHomePosition(move_group)) {
    RCLCPP_ERROR(LOGGER, "Failed to move to home position.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Moved to home position successfully.");

  geometry_msgs::msg::Pose target_pose;
  // We remove delta so that the arm can reach all the positions asked after.
  float delta = 0.05;
  target_pose.position.x = 0.232 - delta;
  target_pose.position.y = -0.010 + delta;
  target_pose.position.z = 0.334;
  target_pose.orientation.w = 1.0;

  if (!moveToPoseTarget(move_group, target_pose)) {
    RCLCPP_ERROR(LOGGER, "Failed to move to pregrasp position.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Moved to pregrasp position successfully.");

  ////////////////////////////// CARTESAIN MOVEMENT
  // DRAW X MOVEMENT
  std::vector<geometry_msgs::msg::Pose> waypoints;

  RCLCPP_INFO(LOGGER, "Draw an X!");
```

```
  target_pose.position.x += delta;
  target_pose.position.y -= delta;
  waypoints.push_back(target_pose);

  target_pose.position.y += delta;
  waypoints.push_back(target_pose);

  target_pose.position.x -= delta;
  target_pose.position.y -= delta;
  waypoints.push_back(target_pose);

  target_pose.position.y += delta;
  waypoints.push_back(target_pose);

  moveit_msgs::msg::RobotTrajectory trajectory;
  const double jump_threshold = 0.0;
  const double eef_step = 0.01;

  if (!performCartesianPath(move_group, waypoints, jump_threshold, eef_step,
                            trajectory)) {
    RCLCPP_ERROR(LOGGER, "Failed to execute approach.");
    return 1; // Exit if the movement failed
  }
  RCLCPP_INFO(LOGGER, "Approach executed successfully.");

  rclcpp::shutdown();
  return 0;
}
```

- End Solution for Exercise 2.3 -

2.8 Kinematics Plugin

You may have noticed that sometimes, the robot performs unnecessarily complicated trajectories when planning to an end effector pose.
When planning to an end effector pose, MoveIt2 generates the trajectory by computing the IK. It uses a **Kinematics Plugin** for this.
The default IK plugin for MoveIt is the `KDLKinematicsPlugin`. The MoveIt2 Setup Assistant automatically configures this plugin. You can find it in the `kinematics.yaml` file:

In [ ]:

```
kdl_kinematics_plugin/KDLKinematicsPlugin
```

- **Change the Kinematics Plugin used by MoveIt2**.

- Open the `/home/user/ros2_ws/src/limo_moveit_config/config/kinematics.yaml` file and replace its contents with the following:

kinematics.yaml

In [ ]:

```
ur_manipulator:
  #kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver: lma_kinematics_plugin/LMAKinematicsPlugin
  kinematics_solver_search_resolution: 0.0050000000000000001
  kinematics_solver_timeout: 0.0050000000000000001
```

As you can see, you have reviewed the `KDLKinematicsPlugin` plugin and added the `LMAKinematicsPlugin`.

- Recompile everything to ensure that the new Kinematics Plugin is used:

Execute in Shell

In [ ]:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

- Restart MoveIt2 for the changes to take effect.
- Also, restart the simulation to start from a fresh position.

Execute in Shell #1

In [ ]:

```
ros2 run gazebo_spawn_tools spawn_entity.py -file ~/ros2_ws/src/grasp_box.urdf -x 0.257792 -y 0 -z 0.113 -entity grasp_box
```

Execute in Shell #1

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config move_group.launch.py
```

Execute in Shell #2

In [ ]:

```
cd ~/ros2_ws
source install/setup.bash
ros2 launch limo_moveit_config moveit_rviz.launch.py
```

Execute in Shell 3

In [ ]:

```
cd ~/ros2_ws
```

```
source install/setup.bash
ros2 run attach_gazebo attach_client_v2_node joint6_flange grasp_box -0.12
```

Execute in Shell 4

In [ ]:

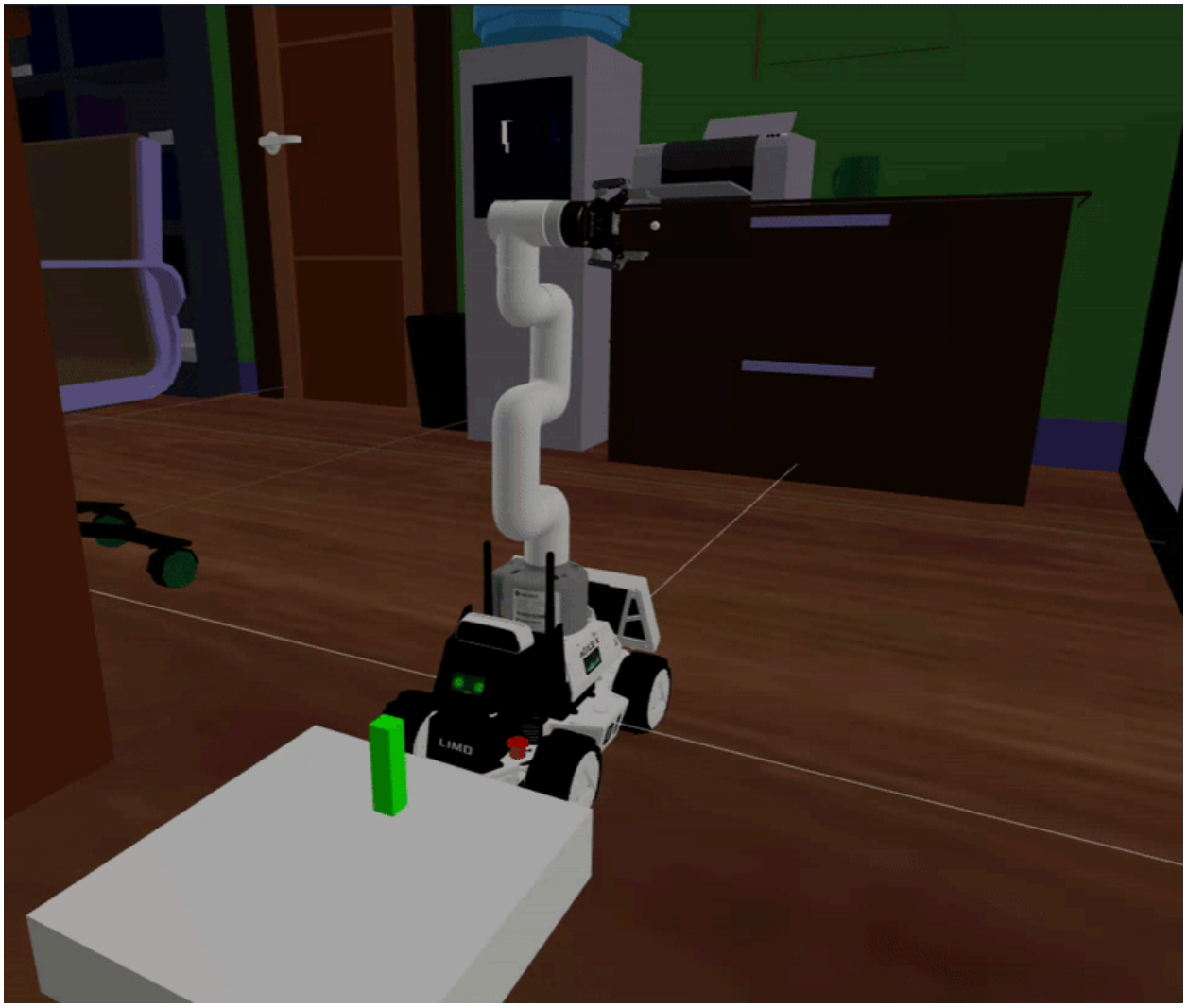
```
cd ~/ros2_ws
source install/setup.bash
ros2 launch moveit2_scripts approach_retreat.launch.py
```

NOTE: Remember the command to open the gripper manually:

Execute in Shell 4

In [ ]:

```
ros2 action send_goal /gripper_controller/gripper_cmd control_msgs/action/GripperCommand
"command:
  position: 0.0
  max_effort: 0.0
"
```



You will likely not see significant differences in the plugins' performance, as they are similar in the end. However, you need to know that you can create your own custom Kinematics Plugin for your robot, which will significantly impact the IK solutions obtained. We hope to cover this topic in more depth in future course updates.

ENGLISH PROOFREAD